autolisp-benchmark — Development Plan

clautolisp

July 2, 2026

Contents

1 Goal

Give the project a way to answer "how fast is this?" for AutoLISP, and "how does clautolisp compare to AutoCAD and BricsCAD?" — with a single body of portable AutoLISP source that runs on all three engines on the same hardware and reports comparable throughput per operation class.

2 Why a separate subproject

The benchmark is neither a conformance test (that is `autolisp-test`) nor part of the runtime. Like `autolisp-test` it is AutoLISP source loaded

into an engine, so it follows the same subproject shape: `harness/` + workload files + `documentation/` tracks + a Makefile that drives it through `clautolisp` and, via `alfe`, through real CADs.

3 Design decisions

3.1 One workload per class, time-boxed

The prompt asked for each operation class to run for about five seconds. Rather than many micro-benchmarks per class, each class is a single representative workload run for `*bench-seconds*`. This keeps the report small and makes "time per category and globally" fall out directly. Additional workloads per class can be added later; the registry and reporting already group by class.

3.2 Amortised dispatch via batches

A workload takes a `reps` count and loops internally, so the cost of calling across the AutoLISP `apply` boundary is negligible against the measured work. The engine calibrates the batch to ~50 ms so timer granularity and loop overhead do not distort the result.

3.3 Throughput, not elapsed, is the comparison axis

Every class is time-boxed to the same budget, so elapsed time is nearly constant and uninformative across engines; iterations/second is what varies and what the report emphasises.

3.4 The clock (the one hard portability problem)

The natural AutoLISP timers — `(getvar "MILLISECS")` and `(getvar "DATE")` — are live on AutoCAD and BricsCAD, and clautolisp's 1.3.x line exposes them as live, read-only clocks too (MILLISECS / DATE / CDATE sourced from the runtime clock rather than the static zero cells of the raw sysvar catalogue). Without that, an in-AutoLISP timer would read a constant on clautolisp and every elapsed interval would be zero, defeating the whole comparison. The harness prefers MILLISECS and falls back to DATE, so it also works on any engine that exposes only one of them.

3.5 Detection via `(ver)`

`(ver)` is the reliable cross-engine self-identifier. The mock host's PRODUCT sysvar reads "BricsCAD" even under clautolisp, so it cannot be used to recognise the engine; `(ver)` can.

3.6 CAD workload uses entity primitives only

`entmake` / `entlast` / `entget` / `entmod` / `entdel` run identically on clautolisp's in-memory drawing database and on real CAD drawings. `(command …)` is avoided: clautolisp does not implement it, and it would drag in an interactive command loop. The entity name is taken from `(entlast)` rather than the `(entmakex)` return value to sidestep `issues/open/entmakex-returns-list.issue` (clautolisp's `entmakex` returns the DXF list instead of an ename); `(entlast)` returns an ename on every target.

4 Roadmap

See `PLAN.md` for the live backlog. Highlights:

- Machine-readable result file + a `compare` helper for engine-vs-engine ratio tables.
- Interpreted-vs-compiled and `vlisp-compile` optimisation-level axes on real CADs (out of scope on clautolisp).
- A `command`-loop CAD workload once clautolisp implements COMMAND.

5 Relationship to other subprojects

- `clautolisp` — provides the engine and the live timer sysvars the harness relies on.
- `autolisp-front-end (alfe)` — the driver that loads the suite into BricsCAD / AutoCAD for the cross-engine comparison.
- `autolisp-test` — the structural template (harness + explicit file manifest + documentation tracks) this subproject follows.